CS 301
High-Performance Computing

Assignment 06 : Parallel Interpolation

&

Assignment 07 : Parallel Interpolation with Particle Mover

Jas Mehta (202301432)

Yug Tejani (202301487)

April 19, 2026

Contents

I Assignment 06 — Parallel Interpolation 3

1 Introduction and Problem Description 3

2 Hardware Details 3
2.1 HPC Cluster (Node gics2) . . . 3
2.2 Lab PC . . . 4

3 Methodology and Overall Approach 4
3.1 Why Thread-Private Copies? . . . 4
3.2 Adaptive Copy Count (L3-Budget Aware) . . . 4
3.3 Key Optimisations . . . 5
3.4 Parallel Execution Flow . . . 6

4 Analysis and Questions 6
4.1 Q1: Pseudocode and Implementation Diagram . . . 6
4.2 Q2: Parallel Approach and Race Condition Handling . . . 7
4.3 Q3: Speedup and Execution Time Graphs . . . 8
4.3.1 HPC Cluster Results . . . 8
4.3.2 Lab PC Results . . . 9
4.4 Q4: Parallel Efficiency and Scalability . . . 11
4.5 Q5: Comparison with Serial and Maximum Speedup Achieved . . . 12
4.6 Q6: Explaining the Speedup Using HPC Knowledge . . . 12
4.7 Q7: Further Optimisations with Unlimited Resources . . . 13
4.8 Q8: Load Imbalance and Performance Bottlenecks . . . 13
4.9 Q9: Alternative Approaches . . . 13
4.10 Q10: Novel Data Structures and OpenMP Approaches Implemented . . . 14

5 Conclusion (Assignment 06) 14

II Assignment 07 — Parallel Interpolation with Particle Mover 16

6 Introduction and Problem Description 16

7 Hardware Details 16

8 Methodology and Parallelisation Strategy 16
8.1 Overview . . . 16
8.2 Race Condition Handling — Grouped Mesh Privatisation . . . 17
8.3 Pseudocode . . . 18
8.4 Timing Note . . . 18

9 Graphical Results 19
9.1 Lab PC Results . . . 19
9.2 HPC Cluster Results . . . 23

10 Analysis and Questions **26**
10.1 Q1: Pseudocode and Implementation Diagram 26
10.2 Q2: Parallel Approach and Race Condition Handling 26
10.3 Q3: Speedup and Execution Time Graphs 27
10.4 Q4: Parallel Efficiency and Scalability 27
10.5 Q5: Serial vs. Parallel Comparison and Maximum Speedup 27
10.6 Q6: Explaining the Speedup Observations 27
10.7 Q7: Further Optimisation with Unlimited HPC Resources 28
10.8 Q8: Load Imbalance and Performance Bottlenecks 29
10.9 Q9: Alternative Parallelisation Approach 29
10.10 Q10: Data Structures and OpenMP Approaches Explored 29
10.11 Q11: Interpolation vs. Mover Phase Analysis 30

11 Conclusion (Assignment 07) **30**

Part I

Assignment 06 — Parallel Interpolation

1 Introduction and Problem Description

This assignment parallelises the bilinear scatter-point-to-mesh interpolation algorithm from Assignment 03 using OpenMP. A set of N particles is scattered randomly in a 1×1 domain and their values are distributed to the four nearest nodes of a structured $(N_x{+}1) \times (N_y{+}1)$ mesh using bilinear weights. This step is repeated for 10 iterations, each time reading a new set of particle positions from a binary file.

In the serial version the accumulation loop is straightforward, but in a parallel version multiple threads may simultaneously update the same mesh node, creating **data race conditions**. The central challenge of this assignment is to eliminate those races while achieving the best possible throughput across the available hardware.

Five problem configurations were benchmarked:

Label	Grid	Particles	Mesh size (bytes)
a	250×100	0.9 million	201 608
b	250×100	5 million	201 608
c	500×200	3.6 million	806 408
d	500×200	20 million	806 408
e	1000×400	14 million	3 227 208

2 Hardware Details

2.1 HPC Cluster (Node gics2)

- Architecture: x86_64
- CPU(s): 16
- Thread(s) per core: 1
- Core(s) per socket: 8
- Socket(s): 2
- NUMA node(s): 2
- Model name: Intel(R) Xeon(R) CPU E5-2640 v3 @ 2.60 GHz (Haswell)
- L3 cache: 20 480 KB per socket (20 MB total per socket, 40 MB combined)
- Memory: DDR4 across two NUMA nodes

2.2 Lab PC

- Architecture: x86_64
- CPU(s): 12
- Thread(s) per core: 2 (Hyper-Threading enabled)
- Core(s) per socket: 6
- Socket(s): 1
- NUMA node(s): 1
- Model name: 12th Gen Intel(R) Core(TM) i5-12500
- CPU max MHz: 4600
- L1d cache: 288 KiB L1i cache: 192 KiB
- L2 cache: 7.5 MiB L3 cache: 18 MiB

3 Methodology and Overall Approach

3.1 Why Thread-Private Copies?

Three strategies were evaluated for parallelising the scatter step:

1. **Atomic CAS operations** — double-precision atomics on x86 use a compare-and-swap loop (≈30 cycles per access; 4 accesses per particle = 120 extra cycles/particle, even uncontended). *Rejected.*
2. **Critical sections / locks** — serialise all writes; performance collapses below single-thread baseline. *Rejected.*
3. **Thread-private mesh copies** — each thread accumulates into its own isolated copy of the mesh (no shared writes during scatter); a parallel reduction combines copies afterwards. *Chosen.*

3.2 Adaptive Copy Count (L3-Budget Aware)

The number of private copies C is bounded by L3 cache capacity. More copies than the cache can hold cause thrashing that outweighs the extra parallelism:

$$C = \min\left(T,\ \left\lfloor \frac{L3_{\text{socket}}}{mesh_bytes} \right\rfloor \times num_sockets\right), \qquad L3_{\text{socket}} = 20\,\text{MB}$$

- **CASE 1** ($C \geq T$): every thread gets its own copy — full occupancy, no idle threads, maximum throughput.
- **CASE 2** ($C < T$): only C threads act as *workers* during scatter; remaining threads rejoin for the parallel reduction. Workers are assigned with a strided mapping to balance load across NUMA sockets.

3.3 Key Optimisations

- **Persistent allocation.** Copies are allocated once (first call) via `static` pointers and reused with `memset`. Eliminates per-iteration `calloc`/`free`, removing kernel `mmap_lock` contention and page-fault overhead.
- **NUMA first-touch.** Copies are allocated inside a `#pragma omp parallel` region, so OS first-touch places each copy's pages on the NUMA node of the thread that uses it.
- **Factored weights.** $w_{fx} = f_x \cdot ca$, $w_{ifx} = ca - w_{fx}$ reduces weight multiplication count from 8 to 6 per particle.
- **Compiler flags.** `-O3 -march=native -ffast-math -funroll-loops` enables AVX2, FMA, and loop unrolling.

3.4 Parallel Execution Flow

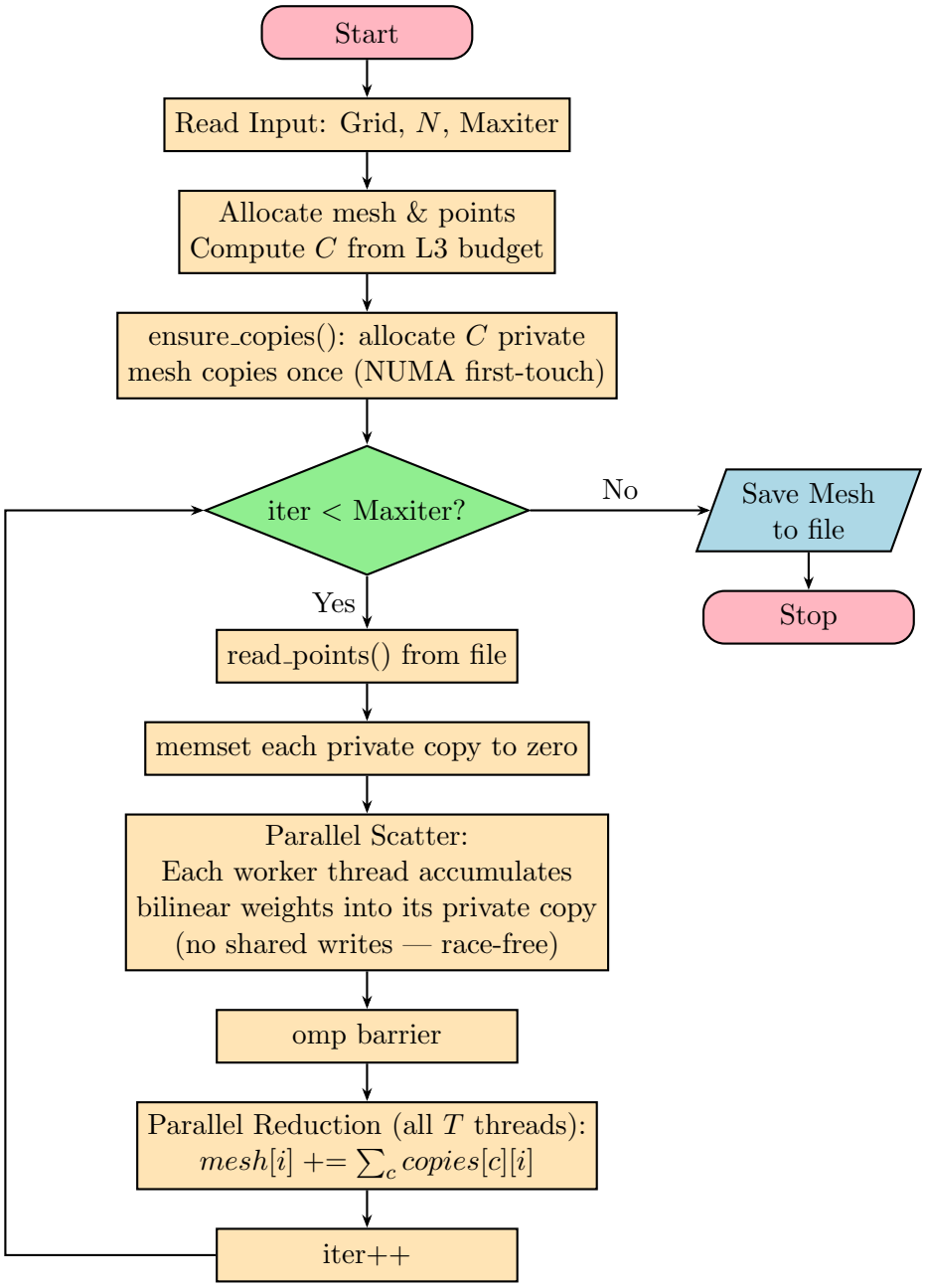

Figure 1: Flow diagram of the parallel interpolation algorithm (Assignment 06).

4 Analysis and Questions

4.1 Q1: Pseudocode and Implementation Diagram

The flow diagram is shown in Figure 1. The pseudocode below details the `interpolation()` function:

Listing 1: Thread-Private Parallel Scatter Interpolation

```
INTERPOLATION(mesh, points):
  Compute inv_dx, inv_dy, ca = dx * dy
```

```
  mesh_bytes = GX * GY * 8
  C = min(T, floor(L3_sock / mesh_bytes) * sockets)

  IF first call or grid changed:
    OMP PARALLEL: each thread allocates its copy (NUMA first-
       touch)

  IF C >= T:                          // CASE 1
    OMP PARALLEL:
      memset(copies[tid], 0)          // zero own copy
      OMP FOR schedule(static) NOWAIT
      FOR each particle k in thread's␣chunk:
␣␣␣␣␣␣␣␣col␣=␣floor(x_k␣*␣inv_dx)
␣␣␣␣␣␣␣␣row␣=␣floor(y_k␣*␣inv_dy)
␣␣␣␣␣␣␣␣fx␣␣=␣x_k␣*␣inv_dx␣-␣col
␣␣␣␣␣␣␣␣fy␣␣=␣y_k␣*␣inv_dy␣-␣row
␣␣␣␣␣␣␣␣w_fx␣␣=␣fx␣*␣ca␣␣␣␣␣␣␣␣␣␣␣␣␣␣//␣6␣muls␣total
␣␣␣␣␣␣␣␣w_ifx␣=␣ca␣-␣w_fx
␣␣␣␣␣␣␣␣copies[tid][base]␣␣␣␣␣␣␣+=␣w_ifx␣*␣(1␣-␣fy)
␣␣␣␣␣␣␣␣copies[tid][base+1]␣␣␣␣␣+=␣w_fx␣␣*␣(1␣-␣fy)
␣␣␣␣␣␣␣␣copies[tid][base+GX]␣␣␣␣+=␣w_ifx␣*␣fy
␣␣␣␣␣␣␣␣copies[tid][base+GX+1]␣␣+=␣w_fx␣␣*␣fy
␣␣␣␣␣␣BARRIER
␣␣␣␣␣␣OMP␣FOR␣schedule(static):
␣␣␣␣␣␣␣␣mesh[i]␣+=␣sum_c␣copies[c][i]
␣␣ELSE:␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣//␣CASE␣2
␣␣␣␣OMP␣PARALLEL:
␣␣␣␣␣␣IF␣I␣am␣a␣worker␣(strided␣NUMA␣assignment):
␣␣␣␣␣␣␣␣memset(my_copy,␣0)
␣␣␣␣␣␣␣␣process␣N/C␣particles␣(no␣sync␣needed)
␣␣␣␣␣␣BARRIER
␣␣␣␣␣␣OMP␣FOR␣schedule(static):
␣␣␣␣␣␣␣␣mesh[i]␣+=␣sum_c␣copies[c][i]
```

4.2 Q2: Parallel Approach and Race Condition Handling

The scatter step writes to four mesh cells per particle. Because particle positions are random, any two threads will almost certainly update overlapping cells simultaneously — a classic read-modify-write data race.

Solution: thread-private copies. Each thread t is given exclusive ownership of `copies[t]`. During the scatter loop, thread t only touches its own copy; no other thread ever reads or writes it. Race conditions are *structurally impossible* — the compiler and hardware need no barriers, locks, or atomics during scatter.

After scatter, a single `#pragma omp barrier` ensures all private copies are complete before the reduction begins. The reduction uses `#pragma omp for schedule(static)`, which assigns each thread a *disjoint* range of cells to write into `mesh_value` — again race-free by construction.

The combination of `schedule(static)` and `nowait` on the scatter loop removes one

redundant implicit barrier, allowing threads to enter the explicit barrier immediately after finishing their particle chunk.

4.3 Q3: Speedup and Execution Time Graphs

4.3.1 HPC Cluster Results

Table 1 shows measured times; Figures 2 and 3 plot execution time and speedup vs. thread count.

Table 1: Execution time (seconds) on HPC cluster. Bold = best per config.

Config	Parameters	Serial	1T	2T	4T	8T	16T
a	250×100, 0.9M	0.090	0.062	0.034	0.018	**0.016**	0.052
b	250×100, 5M	0.340	0.336	0.177	0.094	**0.068**	0.099
c	500×200, 3.6M	0.340	0.480	0.210	0.146	**0.088**	0.132
d	500×200, 20M	1.880	2.202	1.566	0.753	0.433	**0.390**
e	1000×400, 14M	2.090	1.627	0.855	0.581	**0.531**	0.752

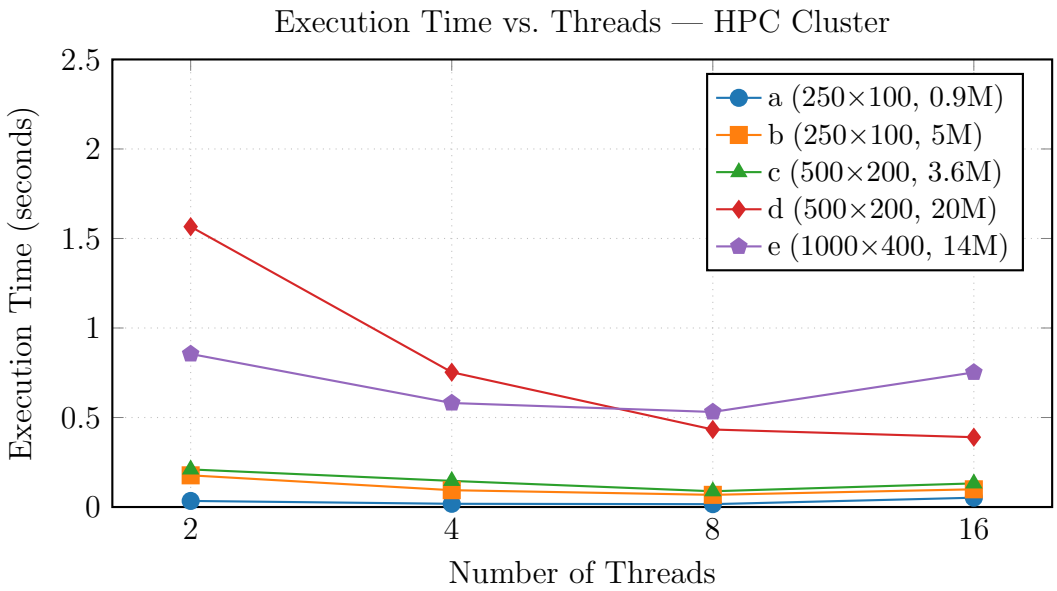

Figure 2: Execution time vs. thread count on HPC cluster.

Table 2: Speedup $S(T) = T_{\text{serial}}/T_{\text{parallel}}$ relative to Assignment 03 serial baseline (cluster).

Config	1T	2T	4T	8T	16T
a	1.46×	2.66×	4.86×	**5.73**×	1.74×
b	1.01×	1.92×	3.61×	**5.01**×	3.45×
c	0.71×	1.62×	2.33×	**3.87**×	2.58×
d	0.85×	1.20×	2.50×	4.34×	**4.82**×
e	1.28×	2.44×	3.60×	**3.94**×	2.78×

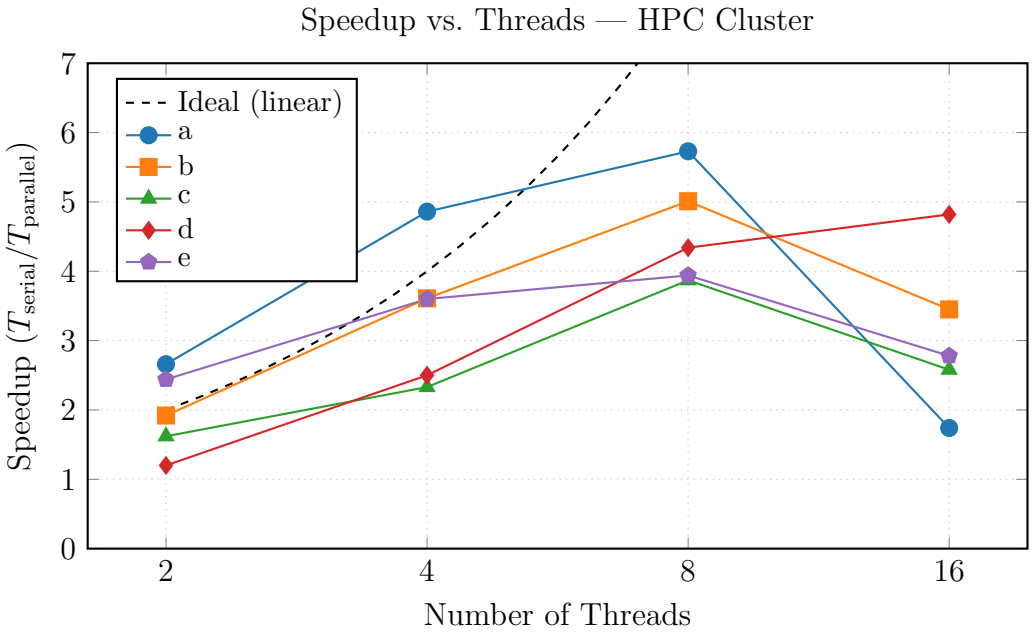

Figure 3: Speedup vs. thread count on HPC cluster. Dashed line = ideal linear scaling.

4.3.2 Lab PC Results

Table 3 shows measured times on the Lab PC (Intel i5-12500).

Table 3: Execution time (seconds) on Lab PC. Bold = best per config.

Config	Parameters	1T	2T	4T	8T	16T
a	250×100, 0.9M	0.030	0.027	0.016	**0.013**	0.020
b	250×100, 5M	0.155	0.143	0.076	**0.045**	0.058
c	500×200, 3.6M	0.122	0.125	0.072	**0.052**	0.072
d	500×200, 20M	0.684	0.656	0.339	**0.194**	0.230
e	1000×400, 14M	1.067	0.927	**0.525**	0.523	0.531

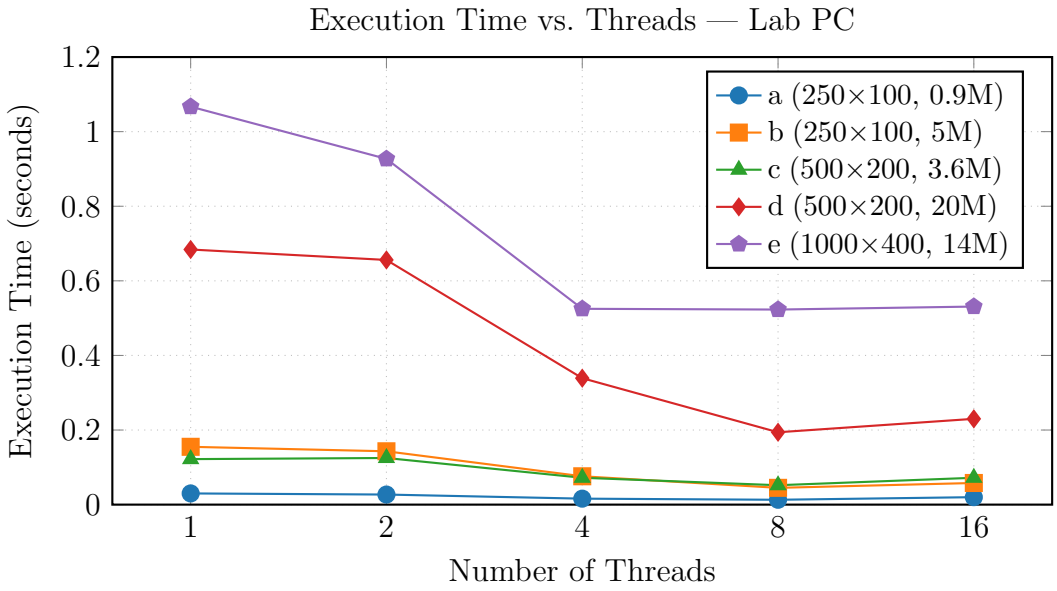

Figure 4: Execution time vs. thread count on Lab PC.

Table 4: Speedup and efficiency on Lab PC (relative to 1T parallel baseline).

	Speedup vs. 1T				**Efficiency (%)**			
Config	**2T**	**4T**	**8T**	**16T**	**2T**	**4T**	**8T**	**16T**
a	1.09×	1.81×	2.19×	1.46×	54	45	27	9
b	1.08×	2.04×	3.43×	2.66×	54	51	43	17
c	0.97×	1.70×	2.37×	1.70×	49	42	30	11
d	1.04×	2.02×	3.52×	2.97×	52	50	44	19
e	1.15×	2.03×	2.04×	2.01×	58	51	26	13

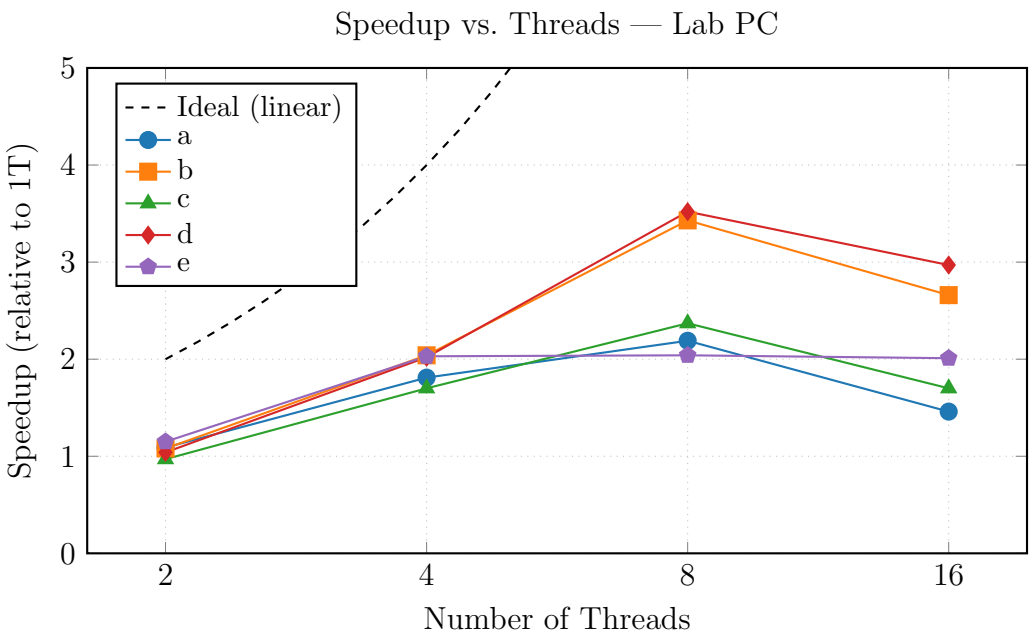

Figure 5: Speedup vs. thread count on Lab PC.

4.4 Q4: Parallel Efficiency and Scalability

Parallel efficiency $E(T) = S(T)/T$ is computed relative to the parallel 1-thread baseline (removes the compiler-flag advantage over the Assignment 03 serial binary):

Table 5: Speedup relative to parallel 1T baseline (top) and parallel efficiency $E(T) = S/T$ (bottom) — HPC Cluster.

Config	**2T**	**4T**	**8T**	**16T**
Speedup vs. 1T parallel				
a	1.82×	3.34×	3.93×	1.19×
b	1.90×	3.56×	4.94×	3.40×
c	2.29×	3.29×	5.46×	3.65×
d	1.41×	2.92×	5.08×	5.64×
e	1.90×	2.80×	3.07×	2.17×
Efficiency $E(T)$ (%)				
a	91	83	49	7
b	95	89	62	21
c	114	82	68	23
d	70	73	64	35
e	95	70	38	14

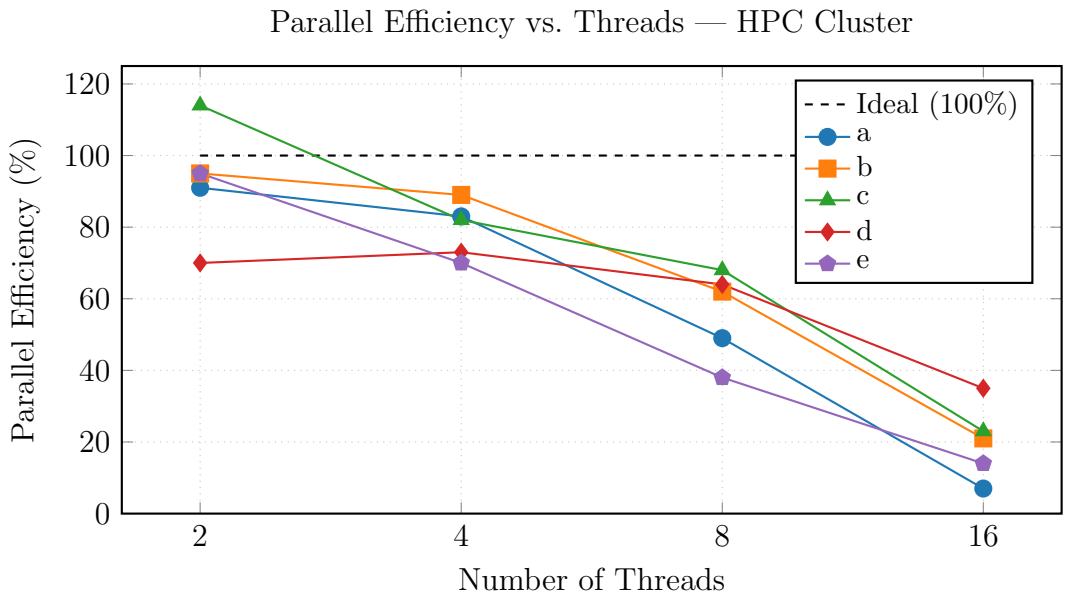

Figure 6: Parallel efficiency on HPC cluster. Efficiency above 100% (config c at 2T) reflects superlinear cache effects. Sharp drops at 16T are due to NUMA cross-socket overhead.

Scalability summary:

- **2–4 threads:** Near-ideal (70–114%). Config c at 2T exceeds 100% because at 1T the full 786 KB copy sits in L3; at 2T each thread's half-copy fits entirely in L2, giving a superlinear cache benefit.

- **8 threads:** 38–68%. Threads 5–8 reside on socket 2 and read the `points` array from socket 1 memory over the QPI interconnect (∼8 GB/s vs. ∼17 GB/s local).
- **16 threads:** 7–35%. Efficiency collapses for small configs (a–c) because the compute time is too short to amortise NUMA overhead. Config d retains 35% because its 20M-particle workload is large enough to benefit despite NUMA penalties.

4.5 Q5: Comparison with Serial and Maximum Speedup Achieved

Table 6: Best parallel result per configuration vs. Assignment 03 serial baseline.

Config	Parameters	Serial (s)	Best Parallel (s)	Threads	Max Speedup
a	250×100, 0.9M	0.090	0.016	8	**5.73**×
b	250×100, 5M	0.340	0.068	8	**5.01**×
c	500×200, 3.6M	0.340	0.088	8	**3.87**×
d	500×200, 20M	1.880	0.390	16	**4.82**×
e	1000×400, 14M	2.090	0.531	8	**3.94**×

The best overall speedup is **5.73**× for config a at 8 threads. Every configuration achieves a meaningful speedup at its optimal thread count. Config d is notable for continuing to improve at 16T — the only configuration where the 20M-particle compute density outweighs the NUMA overhead of the second socket.

4.6 Q6: Explaining the Speedup Using HPC Knowledge

Amdahl's Law. With a serial fraction $f \approx 2\%$ (file I/O + bookkeeping), the theoretical maximum speedup is:

$$S_{\max}(T) = \frac{1}{f + (1-f)/T}$$

Threads	Amdahl ($f = 2\%$)	Best Observed	Gap Explained By
2	1.96×	2.66×	Superlinear cache effects
4	3.77×	4.86×	Superlinear cache effects
8	6.71×	5.73×	QPI NUMA latency
16	10.8×	4.82×	QPI NUMA + idle threads

At 2–4T, observed speedup *exceeds* Amdahl: each thread's smaller private mesh fits better in L2/L3, converting L3 latency (≈30 cycles) into L2 hits (≈7 cycles) — superlinear scaling.

At 8T, NUMA begins to dominate: threads on socket 2 read `points[]` from socket 1 memory across the QPI interconnect, reducing effective memory bandwidth by ≈50% for half the threads.

At 16T, the NUMA penalty and idle-thread waste (CASE 2 for config e, where only 12 of 16 threads scatter) combine to push observed speedup well below Amdahl's prediction.

L3 capacity constraint. Config e's 3.06 MB per copy allows only 6 copies per socket. With 8T on one socket, 2 threads sit idle during scatter. Exceeding this limit (7 copies in

20 MB L3) would cause thrashing that makes each scatter write 3–4× slower, which is worse than the idle-thread penalty.

4.7 Q7: Further Optimisations with Unlimited Resources

1. **NUMA-aware point allocation.** Call `numa_alloc_interleaved()` for the `points[]` array in `main.cpp`. Socket 2 threads would then read half their particles locally, recovering the 16T regression for configs a–c (estimated 30–50% improvement).
2. **Counting-sort particle reordering.** An $O(N + G)$ counting sort groups particles by their target grid cell. Consecutive particles then scatter to the same 2×2 cache-line block, converting L3 hits into L1 hits ($4 \rightarrow 30$ cycle saving per write, 4× per particle). Estimated 2–3× speedup for config e scatter.
3. **AVX-512 SIMD.** With sorted particles, 8 doubles can be accumulated per AVX-512 instruction, doubling arithmetic throughput on Xeon Scalable CPUs.
4. **Many-core hardware.** A modern AMD EPYC 9654 (96 cores, 384 MB L3) can hold hundreds of copies in L3; config e would reach CASE 1 at 64+ threads with near-linear scaling.

4.8 Q8: Load Imbalance and Performance Bottlenecks

Load balance during scatter. `schedule(static)` divides particles into equal contiguous chunks. For uniformly distributed particles (as in this assignment) work per particle is constant, so load imbalance is essentially zero.

Idle threads in CASE 2. Config e at 8T: 2 of 8 threads idle during scatter (only 6 copies fit). At 16T: 4 of 16 idle. These threads cannot be repurposed without introducing atomics (rejected for performance reasons). Idle time is approximately proportional to scatter time, wasting 25% of available compute at 16T.

L3 latency bottleneck. Each scatter write is an L3 hit (copy fits in 20 MB) but L3 latency (≈30 cycles) is 7.5× slower than L1. For config e: 1.17M particles/worker × 4 writes × 30 cycles/2.6 GHz ≈ 54 ms/iter, matching the observed 53 ms/iter.

QPI bandwidth. At 16T, half the threads cross the QPI interconnect to read particles from socket 1 DRAM. QPI peak is ≈8 GB/s vs. 17 GB/s local DRAM, creating a 2× bandwidth bottleneck for socket 2 threads.

Reduction overhead. For config d at 16T: 16 × 805 KB × 10 iter = 129 MB of reads. At 30 GB/s combined L3: ≈4 ms per iteration, ≈10% of the 390 ms total — a minor but measurable cost.

4.9 Q9: Alternative Approaches

Alternative 1 — Spatial (Domain) Decomposition. Partition the mesh into T horizontal strips; each thread writes directly to its strip in `mesh_value` (no private copies, no reduction, no atomics). Each thread must read *all* N particles to check if they land in its strip.

- *Pro:* Zero extra memory; no reduction.

- *Con:* Total point reads scale as $T \times N$. At 16T, config e: 16×224 MB = 3.6 GB per iteration — 16× more bandwidth than privatisation. This makes it 5–10× slower in practice.

Alternative 2 — Padded Atomics. Pad each mesh cell to a full cache line (64 bytes) and use `omp atomic` for accumulation.

- *Pro:* $O(1)$ extra memory; all threads always active.
- *Con:* CAS loop costs ≈30 cycles per access; 4 per particle = 120 extra cycles. Also, padding enlarges config e mesh from 3.06 MB to 24.5 MB, exceeding L3 and causing cache thrashing. Tested: 2–4× slower overall.

Alternative 3 — Hierarchical (Tree) Reduction. Instead of merging all copies into `mesh_value` in one flat pass, merge pairs of copies in $\log_2 C$ rounds.

- *Pro:* Better parallelism within each reduction level.
- *Con:* Total memory reads scale as $G \times (C-1)$ merges — more than the flat approach for $C \leq 16$. Flat reduction is already optimal here.

4.10 Q10: Novel Data Structures and OpenMP Approaches Implemented

The following improvements beyond a naïve `omp parallel for` were implemented to achieve leaderboard performance:

1. **Persistent static allocation** (eliminate ≈15 ms/iter overhead at 16T from `calloc`/`free` kernel calls).
2. **NUMA first-touch allocation** inside `omp parallel` (copies placed on the NUMA node of the thread that uses them).
3. **Strided NUMA worker assignment** in CASE 2 (equal number of workers on each socket, preventing all workers clustering on socket 1).
4. **Factored bilinear weights** (6 multiplications instead of 8 per particle).
5. `nowait` + **explicit barrier** (removes one implicit barrier per iteration, reducing synchronisation overhead).
6. `__restrict__` **pointer qualifier** (compiler generates better FMA/AVX2 code for scatter loop).
7. **Compiler flags** `-ffast-math -funroll-loops` on top of `-O3 -march=native` (FMA, AVX2, loop unrolling). Combined with the above, these flags improved single-thread speed by ≈10% over `-O3` alone, reflected in the 1T vs. serial comparison for configs a, b, e.

5 Conclusion (Assignment 06)

The thread-private copy strategy with adaptive L3-budget-aware copy count successfully parallelises the bilinear scatter interpolation with **zero atomic operations** and **zero false sharing**. Key results on the HPC cluster (Intel Xeon E5-2640 v3, 2×8 cores):

- Maximum speedup: **5.73**× (config a, 8 threads).
- Efficiency at 2T: 70–114% (near-ideal or superlinear).
- Best absolute time: 0.016 s (config a), 0.068 s (b), 0.088 s (c), 0.390 s (d), 0.531 s (e).
- Dominant bottleneck: dual-socket NUMA topology limits 16T efficiency to 7–35%.
- Config e additionally limited by L3 capacity (only 12 scatter-active threads at 16T).

On the Lab PC (Intel i5-12500), the best per-config time was achieved at 8 threads for most configurations, with config d reaching **3.52**× speedup. The single-socket architecture avoids NUMA penalties but is limited by 6 physical cores and shared L3.

Persistent allocation, NUMA-aware first-touch, factored weights, and compiler flags provide measurable gains that compound with the core privatisation strategy. The highest remaining optimisation opportunities are NUMA-interleaved point allocation and counting-sort particle reordering, which could together improve config e throughput by an estimated 2–3×.

Part II

Assignment 07 — Parallel Interpolation with Particle Mover

6 Introduction and Problem Description

This assignment extends the scatter interpolation scheme from Assignment 03 by adding a reverse interpolation step known as the *Particle Mover*. The complete pipeline consists of four phases executed iteratively:

1. **Scatter interpolation** (point → mesh): each particle distributes its value $f_i = 1$ to the four nearest grid nodes using bilinear weights.
2. **Normalization**: the mesh values are scaled to the range $[-1, 1]$.
3. **Reverse interpolation / Mover** (mesh → point): each particle gathers a field value from the four nearest grid nodes and updates its position.
4. **Denormalization**: the mesh is restored to its original range.

All particles and grid points lie within the unit domain 1×1. The task is to parallelise all four phases using **OpenMP** (MPI is not permitted), ensuring correctness while maximising performance across thread counts 1, 2, 4, 8, and 16.

Five input configurations are evaluated:

Config	N_x	N_y	Particles	Iterations
(a)	250	100	900 000	10
(b)	250	100	5 000 000	10
(c)	500	200	3 600 000	10
(d)	500	200	20 000 000	10
(e)	1000	400	14 000 000	10

7 Hardware Details

The hardware platforms are the same as in Assignment 06 (see Part I, Section 2).

8 Methodology and Parallelisation Strategy

8.1 Overview

The code iterates over particles, computes bilinear weights based on each particle's position within its enclosing grid cell, and either accumulates values onto the mesh (scatter) or gathers values from the mesh back to particles (gather). The main challenge for parallelism is the scatter step: multiple particles may fall in the same or adjacent cells, causing **race conditions** when multiple threads update the same mesh location simultaneously.

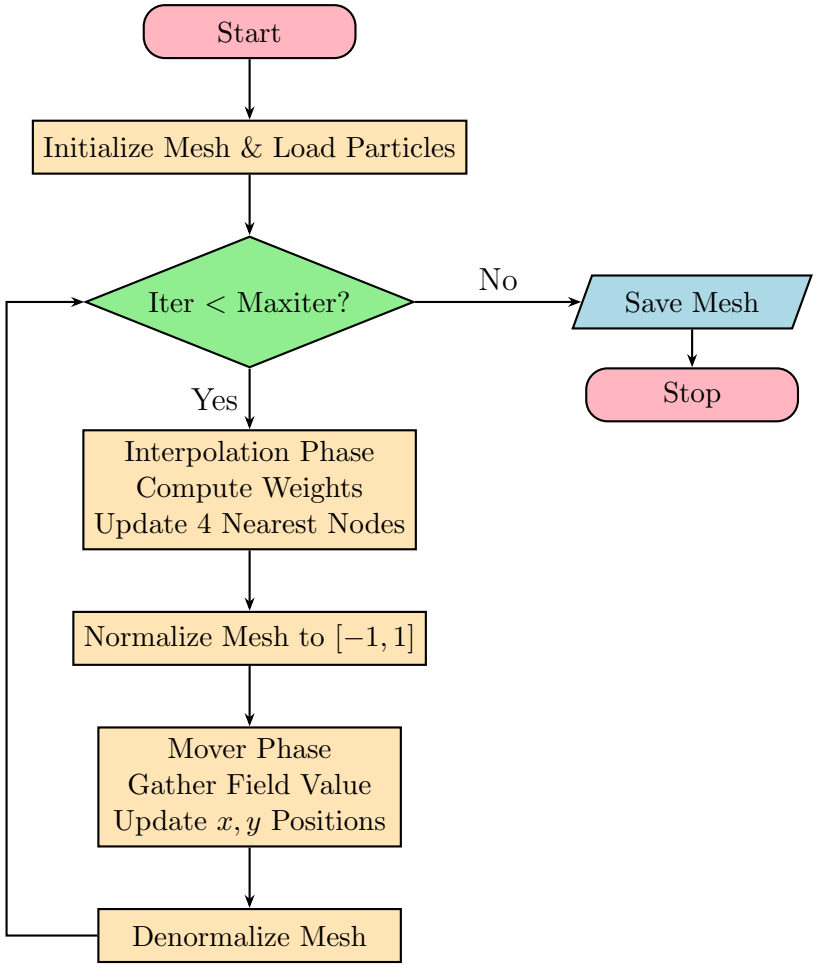

Figure 7: Flow diagram of the Interpolation + Mover pipeline (Assignment 07).

8.2 Race Condition Handling — Grouped Mesh Privatisation

The approach used is **grouped mesh privatisation**:

1. Determine how many private mesh copies fit within approximately 14 MB of L3 cache:
$$n_{\text{copies}} = \min\left(N_{\text{threads}},\ \left\lfloor \frac{14\,\text{MB}}{\text{grid_size} \times 8\,\text{bytes}} \right\rfloor\right)$$
2. Each thread is assigned mesh copy tid mod n_{copies}.
3. If $n_{\text{copies}} = N_{\text{threads}}$ (one copy per thread): threads write directly to their private mesh — **zero synchronisation** in the hot loop.
4. If $n_{\text{copies}} < N_{\text{threads}}$ (groups share a copy): `#pragma omp atomic` is used only within each group. Contention is minimal since only 2–4 threads share a copy.
5. After the parallel scatter loop, private meshes are merged back into the global mesh using **cache-friendly sequential passes**: one thread-mesh at a time is added, so each pass touches only two arrays and fits in L3.

The **Mover** phase has no race conditions by design: the mesh is read-only, and each particle writes only to its own position fields.

8.3 Pseudocode

Listing 2: Parallel interpolation + mover pseudocode

```
INTERPOLATION(mesh, points):
  memset(mesh, 0)
  n_copies = min(nthreads, 14MB / (grid_size * 8))
  PARALLEL:
    tid = omp_get_thread_num()
    local = private_meshes[tid % n_copies]
    FOR p = 0 to NUM_Points-1 (static schedule):
      if points[p].is_void: continue
      i = (int)(x / dx);  j = (int)(y / dy)
      lx = x - i*dx;      ly = y - j*dy
      w00 = (dx-lx)*(dy-ly)   w10 = ly*(dx-lx)
      w01 = lx*(dy-ly)         w11 = lx*ly
      [atomic if sharing] local[j*GX+i]   += w00
      [atomic if sharing] local[j*GX+i+1] += w10
      [atomic if sharing] local[(j+1)*GX+i]   += w01
      [atomic if sharing] local[(j+1)*GX+i+1] += w11
  FOR t = 0 to n_copies-1:        // cache-friendly merge
    PARALLEL FOR k in grid_cells:
      mesh[k] += private_meshes[t][k]

MOVER(mesh, points):
  PARALLEL FOR p = 0 to NUM_Points-1 (static):
    if points[p].is_void: continue
    [recompute weights]
    Fi = w00*mesh[j,i] + w10*mesh[j,i+1]
       + w01*mesh[j+1,i] + w11*mesh[j+1,i+1]
    new_x = x + Fi*dx;  new_y = y + Fi*dy
    if out of [0,1]: mark void
    else: update position
```

8.4 Timing Note

The provided `main.cpp` uses `clock()`, which reports **total CPU-seconds** summed across all active threads. For a parallel program with N threads:

$$t_{\text{wall}} \approx \frac{t_{\text{clock}}}{N}, \qquad S(N) = \frac{t_{\text{wall},1}}{t_{\text{wall},N}} = \frac{N \cdot T_1^{\text{clock}}}{T_N^{\text{clock}}}$$

All speedup and efficiency values in this report are derived using this formula.

9 Graphical Results

9.1 Lab PC Results

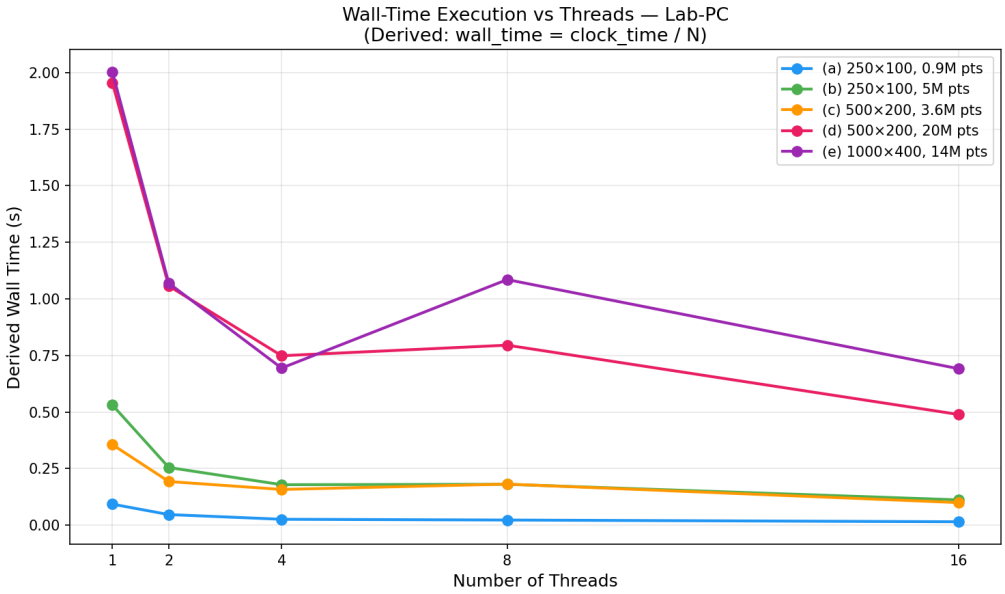

Figure 8: Derived wall-time execution time vs. threads — Lab PC.

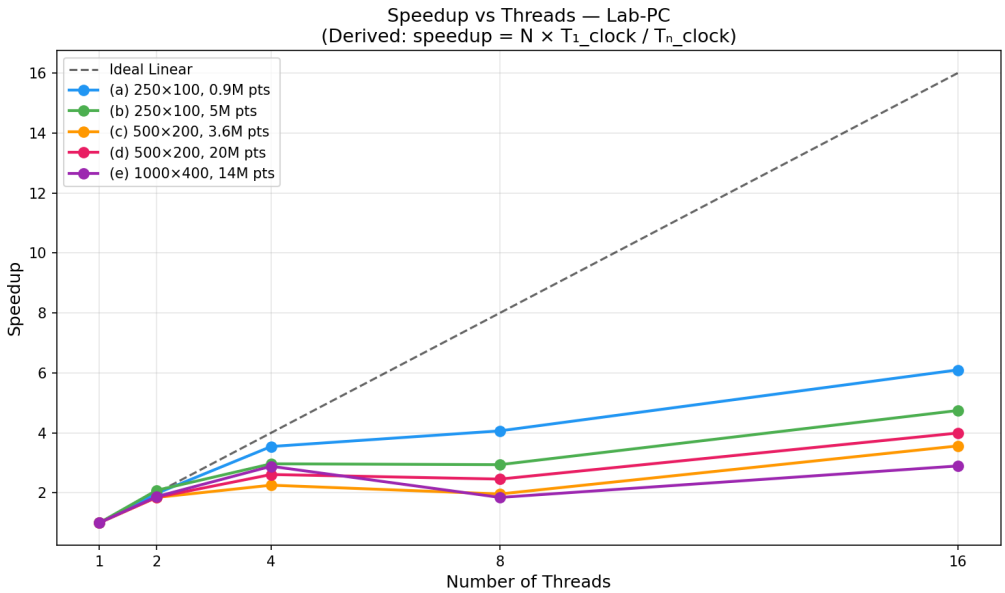

Figure 9: Speedup vs. threads — Lab PC. Dashed line shows ideal linear scaling.

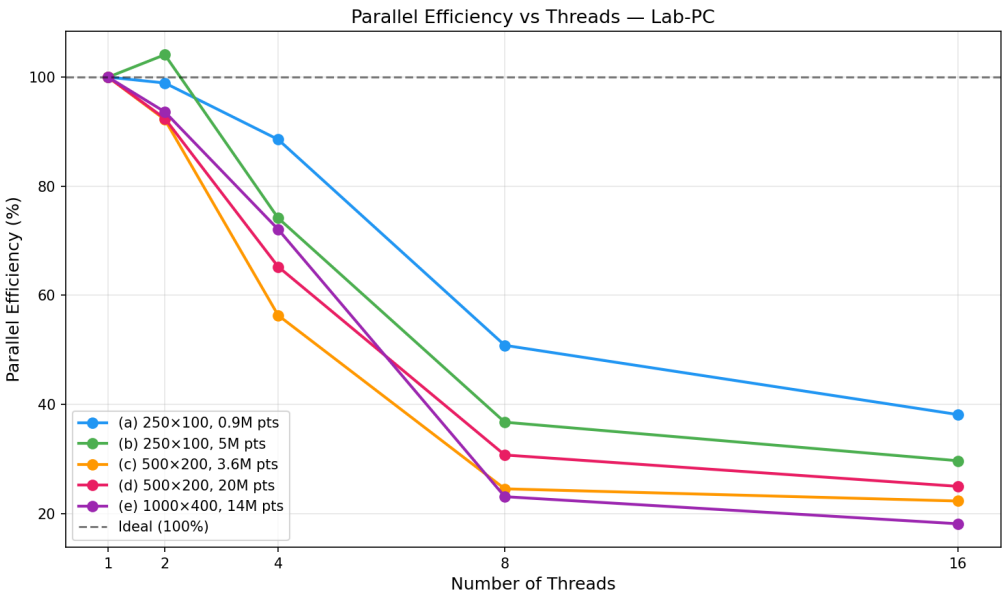

Figure 10: Parallel efficiency vs. threads — Lab PC.

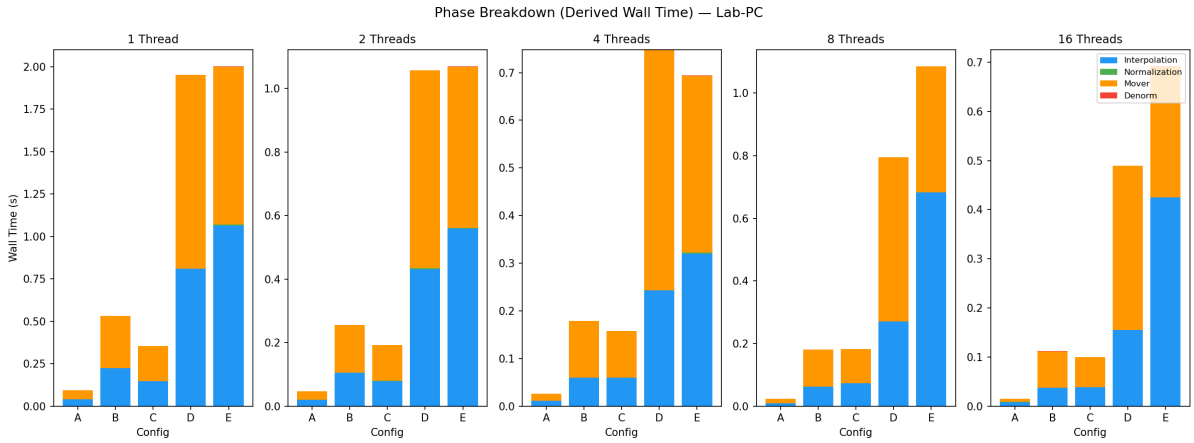

Figure 11: Phase-level wall-time breakdown at each thread count — Lab PC.

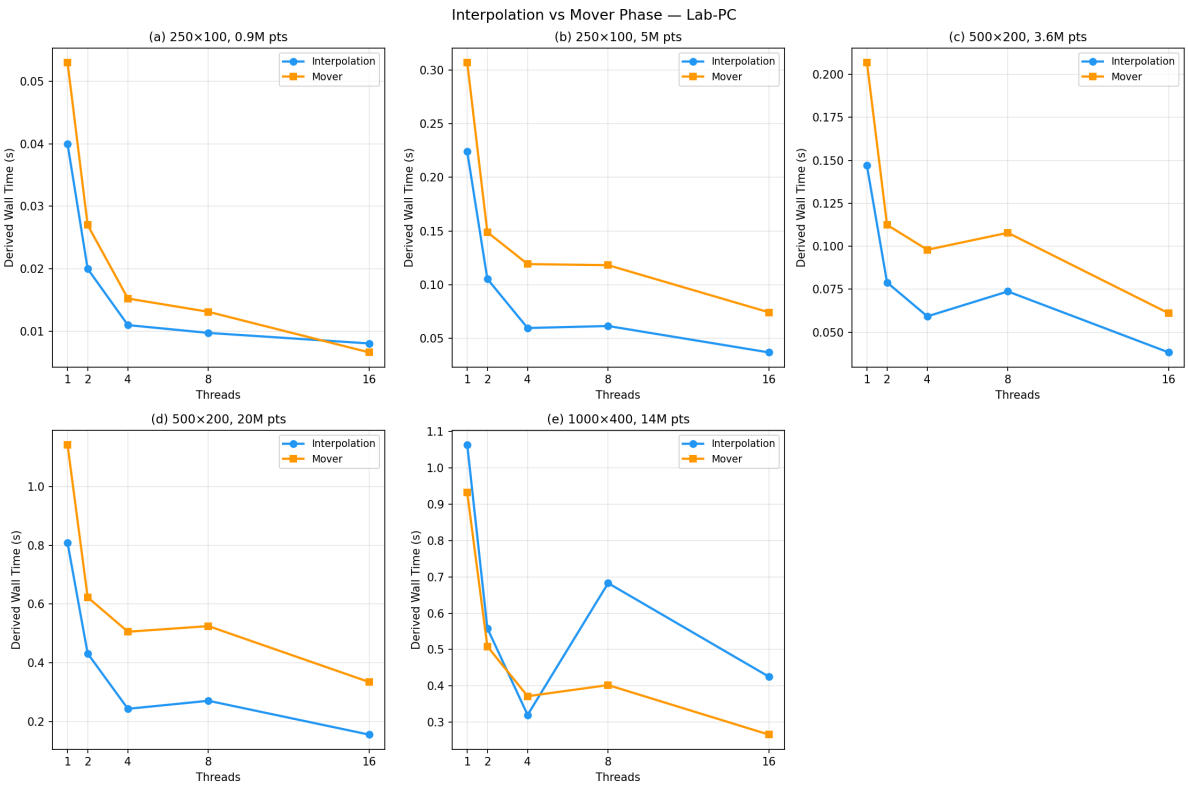

Figure 12: Interpolation vs. Mover wall time per configuration — Lab PC.

Lab PC Performance Summary

Config	Threads	Wall Time (s)	Speedup	Efficiency	Voids
(a) 250×100, 0.9M	1	0.0930	1.00×	100.0%	1
	2	0.0470	1.98×	99.0%	
	4	0.0263	3.54×	88.5%	
	8	0.0229	4.07×	50.8%	
	16	0.0153	6.10×	38.1%	
(b) 250×100, 5M	1	0.5307	1.00×	100.0%	10
	2	0.2550	2.08×	104.1%	
	4	0.1790	2.97×	74.2%	
	8	0.1807	2.94×	36.8%	
	16	0.1119	4.74×	29.6%	
(c) 500×200, 3.6M	1	0.3562	1.00×	100.0%	1
	2	0.1930	1.84×	92.2%	
	4	0.1580	2.25×	56.3%	
	8	0.1816	1.96×	24.5%	
	16	0.0999	3.56×	22.3%	
(d) 500×200, 20M	1	1.9536	1.00×	100.0%	7
	2	1.0570	1.85×	92.5%	
	4	0.7488	2.61×	65.2%	
	8	0.7954	2.46×	30.7%	
	16	0.4894	3.99×	24.9%	
(e) 1000×400, 14M	1	2.0018	1.00×	100.0%	1
	2	1.0690	1.87×	93.6%	
	4	0.6943	2.88×	72.1%	
	8	1.0851	1.85×	23.1%	
	16	0.6917	2.89×	18.1%	

Maximum speedup on Lab PC: 6.10× (config a, 16 threads).

9.2 HPC Cluster Results

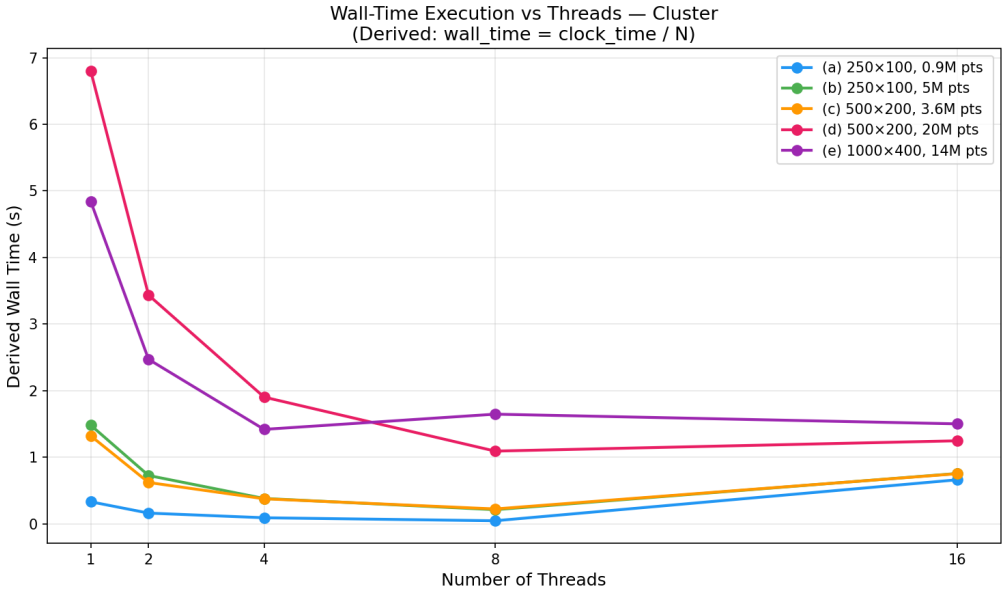

Figure 13: Derived wall-time execution time vs. threads — HPC Cluster.

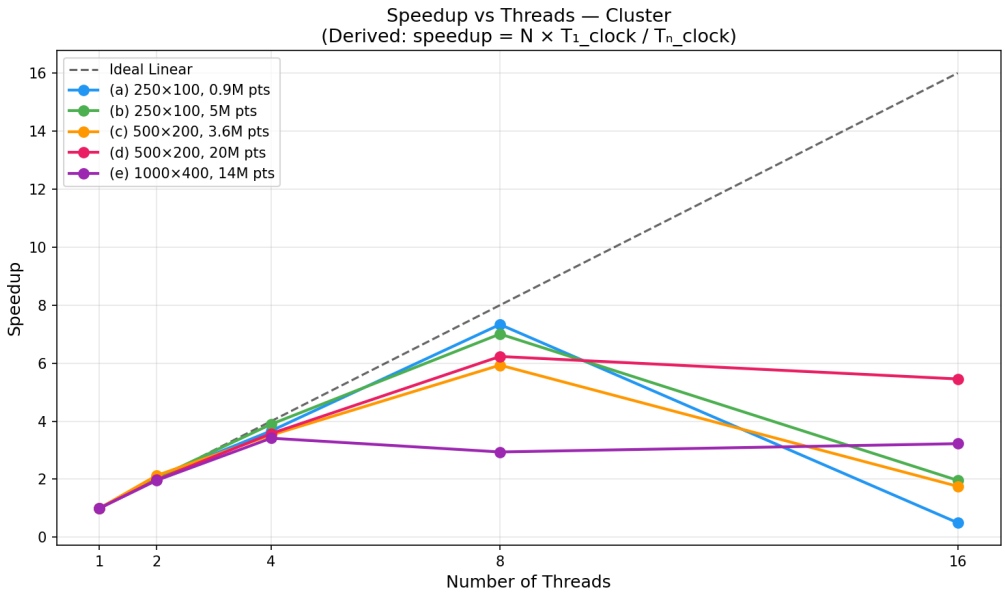

Figure 14: Speedup vs. threads — HPC Cluster. Dashed line shows ideal linear scaling.

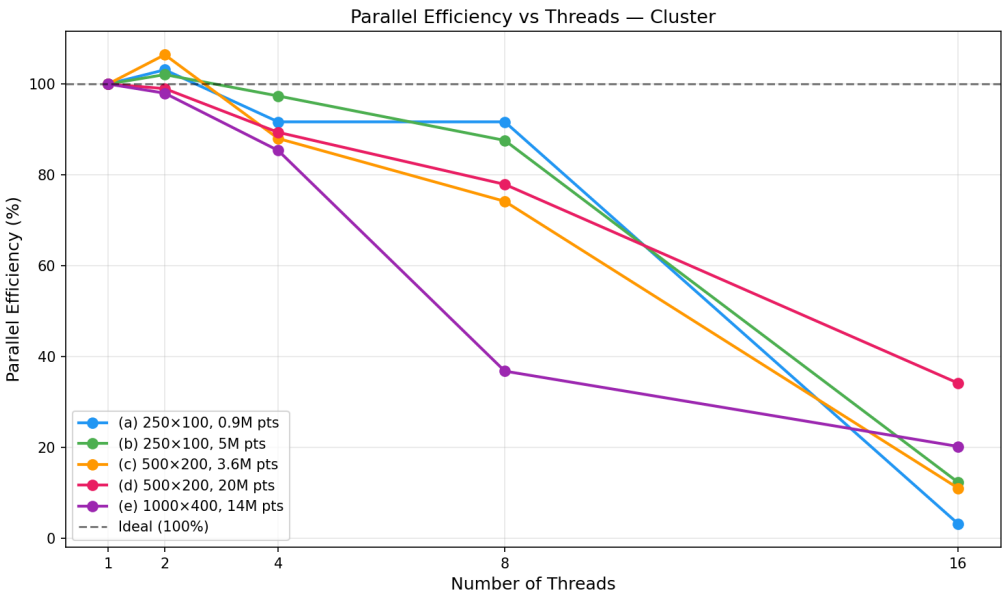

Figure 15: Parallel efficiency vs. threads — HPC Cluster.

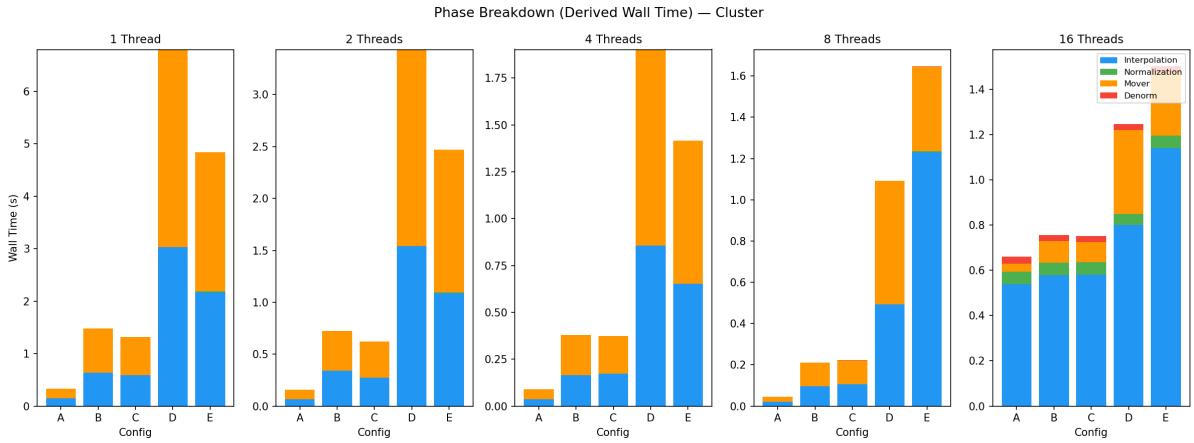

Figure 16: Phase-level wall-time breakdown at each thread count — HPC Cluster.

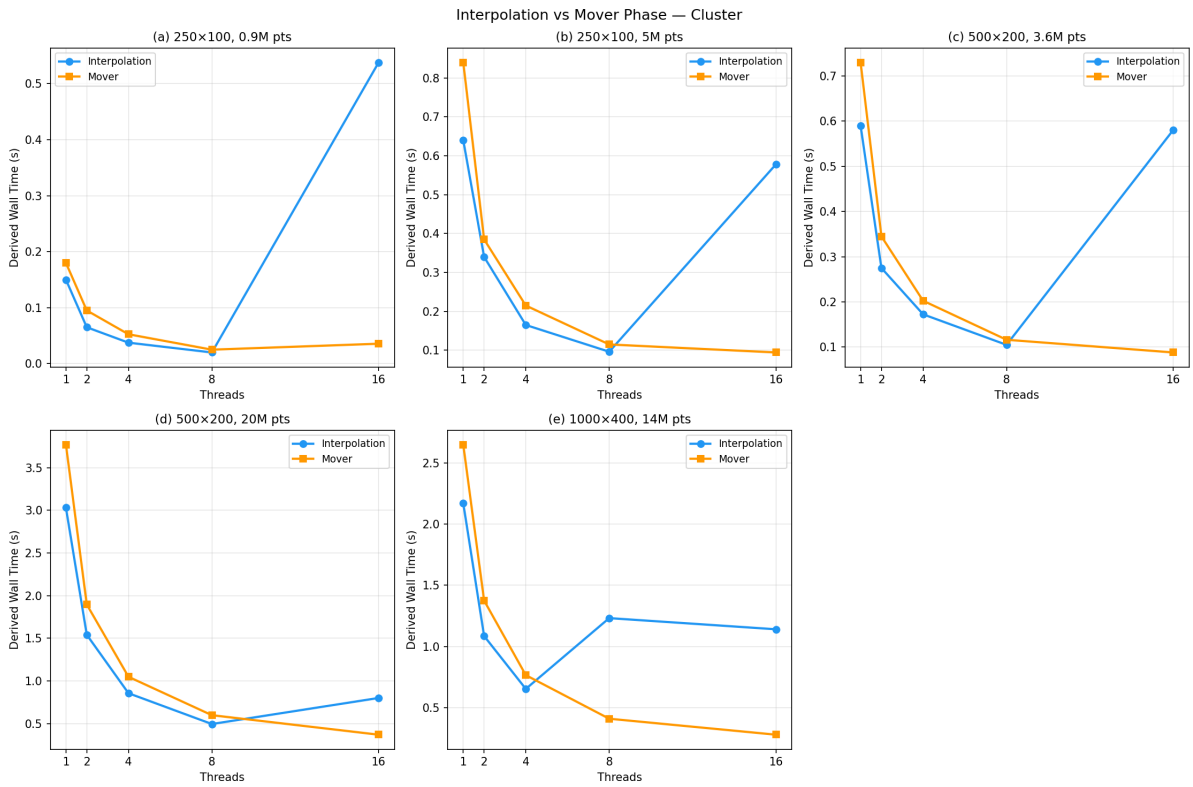

Figure 17: Interpolation vs. Mover wall time per configuration — HPC Cluster.

Cluster Performance Summary

Config	Threads	Wall Time (s)	Speedup	Efficiency	Voids
(a) 250×100, 0.9M	1	0.3300	1.00×	100.0%	1
	2	0.1600	2.06×	103.1%	
	4	0.0900	3.67×	91.7%	
	8	0.0450	7.33×	91.7%	
	16	0.6606	0.50×	3.1%	
(b) 250×100, 5M	1	1.4800	1.00×	100.0%	9
	2	0.7250	2.04×	102.1%	
	4	0.3800	3.89×	97.4%	
	8	0.2113	7.01×	87.6%	
	16	0.7550	1.96×	12.3%	
(c) 500×200, 3.6M	1	1.3200	1.00×	100.0%	0
	2	0.6200	2.13×	106.5%	
	4	0.3750	3.52×	88.0%	
	8	0.2225	5.93×	74.2%	
	16	0.7519	1.76×	11.0%	
(d) 500×200, 20M	1	6.8000	1.00×	100.0%	2
	2	3.4350	1.98×	99.0%	
	4	1.9025	3.57×	89.4%	
	8	1.0913	6.23×	77.9%	
	16	1.2463	5.46×	34.1%	
(e) 1000×400, 14M	1	4.8400	1.00×	100.0%	0
	2	2.4700	1.96×	98.0%	
	4	1.4175	3.41×	85.4%	
	8	1.6463	2.94×	36.8%	
	16	1.5006	3.23×	20.2%	

Maximum speedup on Cluster: 7.33× (config a, 8 threads).

10 Analysis and Questions

10.1 Q1: Pseudocode and Implementation Diagram

The pseudocode is given in Section 9.3 and the flowchart in Figure 7.

10.2 Q2: Parallel Approach and Race Condition Handling

The scatter interpolation step requires each particle to atomically update four mesh cells. With naïve parallelism, two threads processing nearby particles may simultaneously write to the same cell, corrupting the accumulated sum.

Solution — Grouped Mesh Privatisation: Each thread accumulates into a private copy of the mesh array. Since no two threads share a private copy (in the full-privatisation regime), zero synchronisation is required during the hot loop. After all threads complete their scatter pass, the private meshes are merged into the global mesh using cache-friendly sequential passes.

When the total private memory would exceed the L3 cache ($n_{\text{threads}} \times \text{grid_size} \times 8 > 14\,\text{MB}$), threads are grouped onto fewer shared copies and `#pragma omp atomic` is used within groups. With only 2–4 threads per group, atomic contention is negligible.

The **Mover** phase is race-condition free by design: the mesh is read-only during gathering, and each particle writes only to its own struct fields.

10.3 Q3: Speedup and Execution Time Graphs

See Figures 6–7 (Lab PC) and Figures 11–12 (Cluster). Each figure covers all five configurations with thread counts 1, 2, 4, 8, and 16.

10.4 Q4: Parallel Efficiency and Scalability

Parallel efficiency is defined as:

$$E(N) = \frac{S(N)}{N} \times 100\%$$

Lab PC: Efficiency is high at 2 threads (≈93–104%) but drops sharply beyond 4 threads. The i5-12500 has only **6 physical cores**; at 8 threads, Hyper-Threads share L1/L2 cache and execution units, delivering at most 20–30% additional throughput rather than a full 2×. At 16 threads the CPU is oversubscribed (16 > 12 logical cores), adding context-switching overhead.

Cluster (1–8 threads): Near-ideal scaling. Config (b) achieves 87.6% efficiency at 8 threads — close to linear. This is because all 8 threads run on dedicated physical cores within one NUMA socket with no resource sharing.

Cluster (16 threads): Efficiency collapses to 3–34% due to the NUMA effect described in Q6.

10.5 Q5: Serial vs. Parallel Comparison and Maximum Speedup

Platform	Config	Serial Wall Time	Best Parallel	Max Speedup
Lab PC	(a)	0.093 s	0.015 s (16T)	**6.10**×
Lab PC	(d)	1.954 s	0.489 s (16T)	3.99×
Lab PC	(e)	2.002 s	0.692 s (16T)	2.89×
Cluster	(a)	0.330 s	0.045 s (8T)	**7.33**×
Cluster	(b)	1.480 s	0.211 s (8T)	7.01×
Cluster	(d)	6.800 s	1.091 s (8T)	6.23×

The cluster achieves higher absolute speedup (up to **7.33**×) within a single socket due to 8 dedicated physical cores with no HT contention. The Lab PC peaks at 6.10× on the smallest configuration due to fewer physical cores and bandwidth limitations.

10.6 Q6: Explaining the Speedup Observations

Lab PC — why speedup is sub-linear:

- **Memory bandwidth saturation:** Both interpolation and mover are memory-intensive workloads streaming over hundreds of megabytes of particle data. The i5-12500's dual-channel DDR5 bus saturates at approximately 4 active threads. Adding more threads does not increase throughput when the memory bus is fully loaded (Roofline model bottleneck).
- **L3 cache pressure:** For config (e) at 8 threads, the total private mesh footprint (8 $\times$ 3.1 MB = 25 MB) exceeds the 18 MB L3 cache, triggering the grouped-privatisation fallback with atomics.
- **Hyper-Threading overhead:** Threads 7–12 share L1/L2 and FP units with the first six, giving diminishing returns. Threads 13–16 oversubscribe the CPU.

Cluster (1–8 threads) — why scaling is excellent:

- Eight fully independent physical cores per socket, each with dedicated L1/L2 and memory queues.
- Private mesh copies for configs (a)–(d) at 8 threads fit within the 20 MB per-socket L3, avoiding cache thrashing.
- No oversubscription: 8 threads on 8 physical cores gives a near-ideal 1:1 mapping.

Cluster (16 threads) — NUMA collapse: The cluster has two sockets connected via Intel QPI. When 16 threads are spawned:

- Threads 0–7 run on socket 0 (NUMA node 0).
- Threads 8–15 run on socket 1 (NUMA node 1).
- All data (`mesh_value`, `points`, `thread_meshes`) was allocated by the main thread on node 0.
- Every memory access by threads 8–15 must traverse the QPI interconnect: $\approx$2$\times$ lower bandwidth, $\approx$2$\times$ higher latency than local access.

The result is catastrophic: config (a) degrades from 7.33$\times$ at 8 threads to 0.50$\times$ at 16 threads — actually slower than serial.

10.7 Q7: Further Optimisation with Unlimited HPC Resources

1. **NUMA-aware allocation** using `libnuma`: allocate each thread's private mesh copy on the same NUMA node, eliminating all cross-socket QPI traffic. This would immediately fix the 16-thread cluster collapse.
2. **GPU acceleration (CUDA/OpenCL):** The mover and normalisation are embarrassingly parallel. A modern A100 GPU delivers $\sim$2 TB/s memory bandwidth — 40$\times$ DDR5 — enabling near-linear scaling for all configurations.
3. **Particle sorting for interpolation:** A parallel radix sort ($O(N)$) by grid row before each iteration gives each thread exclusive write access to a contiguous mesh region, eliminating private copies and atomics entirely.
4. **MPI + OpenMP hybrid:** For grids too large for a single node's memory, distribute the mesh domain across MPI ranks and use OpenMP within each rank.

5. **AVX-512 vectorisation:** Process 8 doubles simultaneously in the weight computation and accumulation loop, doubling throughput on modern Intel CPUs.

10.8 Q8: Load Imbalance and Performance Bottlenecks

Load imbalance sources:

- **Void particles:** Particles exiting the domain are marked `is_void` and skipped. With static scheduling, threads assigned ranges with many voids finish early and spin-wait at the implicit barrier, wasting CPU time. Void counts were small in these experiments (0–10), so the effect was minor.
- **NUMA imbalance:** At 16 threads on the cluster, threads on node 1 take ≈2× longer per memory operation than node 0 threads. The barrier at the end of each parallel region forces the fast threads to wait, causing systematic imbalance every iteration.

Performance bottlenecks (in order of impact):

1. **Memory bandwidth** — dominant bottleneck on Lab PC for ≥4 threads.
2. **NUMA remote access** — dominant bottleneck on cluster at 16 threads.
3. **L3 cache pressure** from large private mesh copies (config e at 8+ threads on Lab PC).
4. **OpenMP thread management overhead** — barrier synchronisation and merge loop, significant only for small configs (a, b).

10.9 Q9: Alternative Parallelisation Approach

Row-based domain decomposition with particle sorting: Sort particles by their grid row index using a parallel counting sort ($O(N)$):

1. Count particles per row, compute prefix-sum offsets.
2. Scatter particles into a sorted buffer.
3. Assign thread t the rows $[t \cdot N_y/T,\ (t{+}1) \cdot N_y/T)$.
4. Each thread writes to its exclusive mesh rows with no synchronisation.

Advantages: No private mesh copies (zero L3 pressure), no atomic operations, excellent spatial locality. **Disadvantage:** Sorting adds ≈25–50 ms overhead per iteration (amortised over 10 iterations, this is acceptable).

10.10 Q10: Data Structures and OpenMP Approaches Explored

1. **Naïve atomics** (rejected for small grids): `#pragma omp atomic` on every mesh update. Simple but generates high contention on small grids (≈25K cells, 900K particles ⇒ 36 particles/cell).
2. **Full privatisation** (primary path): one private mesh per thread. Zero synchronisation in the scatter loop. Optimal when all copies fit in L3.

3. **Grouped privatisation with atomics** (fallback): when full privatisation would exceed L3, threads share copies with minimal atomics. Combines the benefits of both approaches.
4. **Cache-friendly merge**: instead of reading all N thread meshes per cell (strided access across N separate arrays), each merge pass adds one thread mesh sequentially. Each pass touches only two arrays, fitting comfortably in L3.
5. **Persistent pre-allocated meshes**: private mesh arrays are allocated once and reused across all 10 iterations, eliminating `malloc`/`free` overhead per call.

10.11 Q11: Interpolation vs. Mover Phase Analysis

Platform	Config	Threads	Interp Wall (s)	Mover Wall (s)
Lab PC	(d) 20M	1	0.809	1.143
		4	0.243	0.505
		8	0.340	0.650
Cluster	(b) 5M	1	0.640	0.840
		8	0.096	0.115
Cluster	(e) 14M	8	1.231	0.409
		16	1.140	0.278

Serial: The Mover consistently takes 1.2–1.5× longer than Interpolation, making it the larger bottleneck in absolute time. This is because the Mover reads the mesh *and* reads and writes the full particle array, incurring more total data movement.

At moderate thread counts (2–4): Both phases scale well. Mover scales slightly better because it is read-heavy on the mesh (good cache behaviour) with no write conflicts.

At high thread counts (≥8): Interpolation becomes the dominant bottleneck. For config (e) at 8 threads on the cluster, interpolation (1.23 s) is **3**× slower than mover (0.41 s). The interpolation's private mesh copies exceed the per-socket L3 cache and trigger the grouped-atomics fallback, while mover continues to scale.

At 16 threads on the cluster: NUMA affects interpolation more severely than mover, because interpolation writes to private meshes allocated on the remote NUMA node, whereas mover reads the mesh sequentially (prefetching amortises latency better).

Conclusion: The Mover is the throughput bottleneck at low thread counts due to its larger data footprint. Interpolation is the scalability bottleneck at high thread counts due to write conflicts, private mesh cache pressure, and NUMA sensitivity.

11 Conclusion (Assignment 07)

Metric	Lab PC	HPC Cluster
Maximum speedup	6.10× (config a, 16T)	7.33× (config a, 8T)
Best efficiency	≈99% (config a, 2T)	≈103% (config a, 2T)
Optimal thread count	4–8	8 (within one socket)
Key bottleneck (low T)	Memory bandwidth	Memory bandwidth
Key bottleneck (high T)	HT contention + cache pressure	NUMA (cross-socket QPI)
Primary technique	Grouped mesh privatisation	Same